Unsupervised Learning

Machine

What is Unsupervised Learning?

- Unsupervised learning is a machine learning technique in which models are not supervised using training dataset. Instead, models itself find the hidden patterns and insights from the given data. It can be compared to learning which takes place in the human brain while learning new things.

- It can be defined as:

> *"Unsupervised learning is a type of machine learning in which models are trained using unlabeled dataset and are allowed to act on that data without any supervision."*

- Unsupervised learning cannot be directly applied to a regression or classification problem because unlike supervised learning, we have the input data but no corresponding output data. The goal of unsupervised learning is to **find the underlying structure of dataset, group that data according to similarities, and represent that dataset in a compressed format**.

Example:

1.Suppose the unsupervised learning algorithm is given an input dataset containing images of different types of cats and dogs.
2. The algorithm is never trained upon the given dataset, which means it does not have any idea about the features of the dataset.
3.The task of the unsupervised learning algorithm is to identify the image features on their own.
4. Unsupervised learning algorithm will perform this task by clustering the image dataset into the groups according to similarities between images.

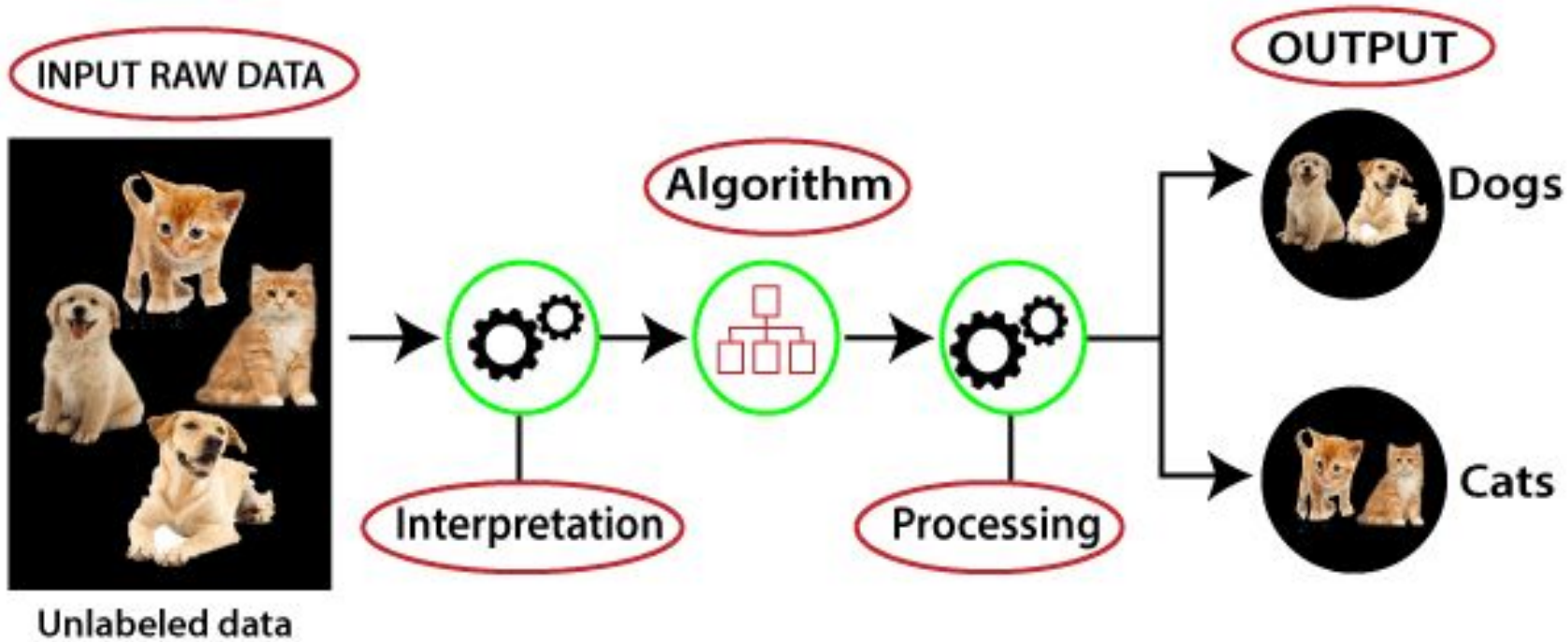
Why use Unsupervised Learning?

Below are some main reasons which describe the importance of Unsupervised Learning:

- Unsupervised learning is helpful for finding useful insights from the data.
- Unsupervised learning is much similar as a human learns to think by their own experiences, which makes it closer to the real AI.
- Unsupervised learning works on unlabeled and uncategorized data which make unsupervised learning more important.
- In real-world, we do not always have input data with the corresponding output so to solve such cases, we need unsupervised learning.

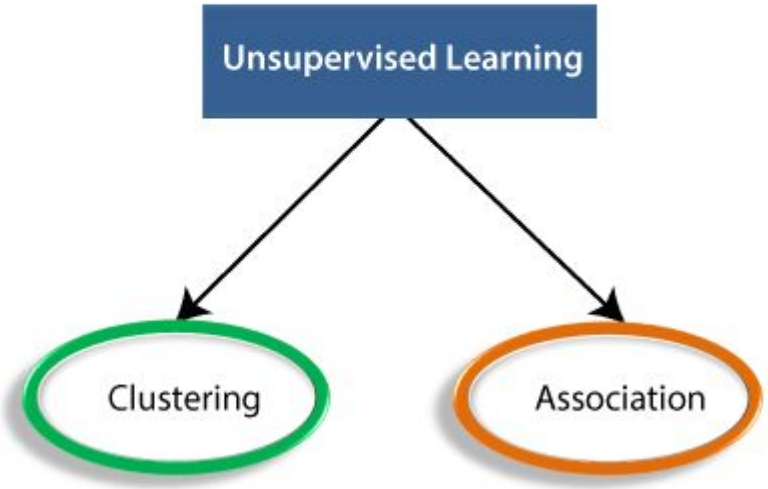
Working of Unsupervised Learning

Working of unsupervised learning can be understood by the below diagram:



Types of Unsupervised Learning Algorithm:

The unsupervised learning algorithm can be further categorized into two types of problems:



- **Clustering**: Clustering is a method of grouping the objects into clusters such that objects with most similarities remains into a group and has less or no similarities with the objects of another group. Cluster analysis finds the commonalities between the data objects and categorizes them as per the presence and absence of those commonalities.

- **Association**: An association rule is an unsupervised learning method which is used for finding the relationships between variables in the large database. It determines the set of items that occurs together in the dataset. Association rule makes marketing strategy more effective. Such as people who buy X item (suppose a bread) are also tend to purchase Y (Butter/Jam) item. A typical example of Association rule is Market Basket Analysis.

Unsupervised Learning algorithms:

Below is the list of some popular unsupervised learning algorithms:

- **K-means clustering**
- **KNN (k-nearest neighbors)**
- **Hierarchal clustering**
- **Anomaly detection**
- **Neural Networks**
- **Principle Component Analysis**
- **Independent Component Analysis**
- **Apriori algorithm**
- **Singular value decomposition**

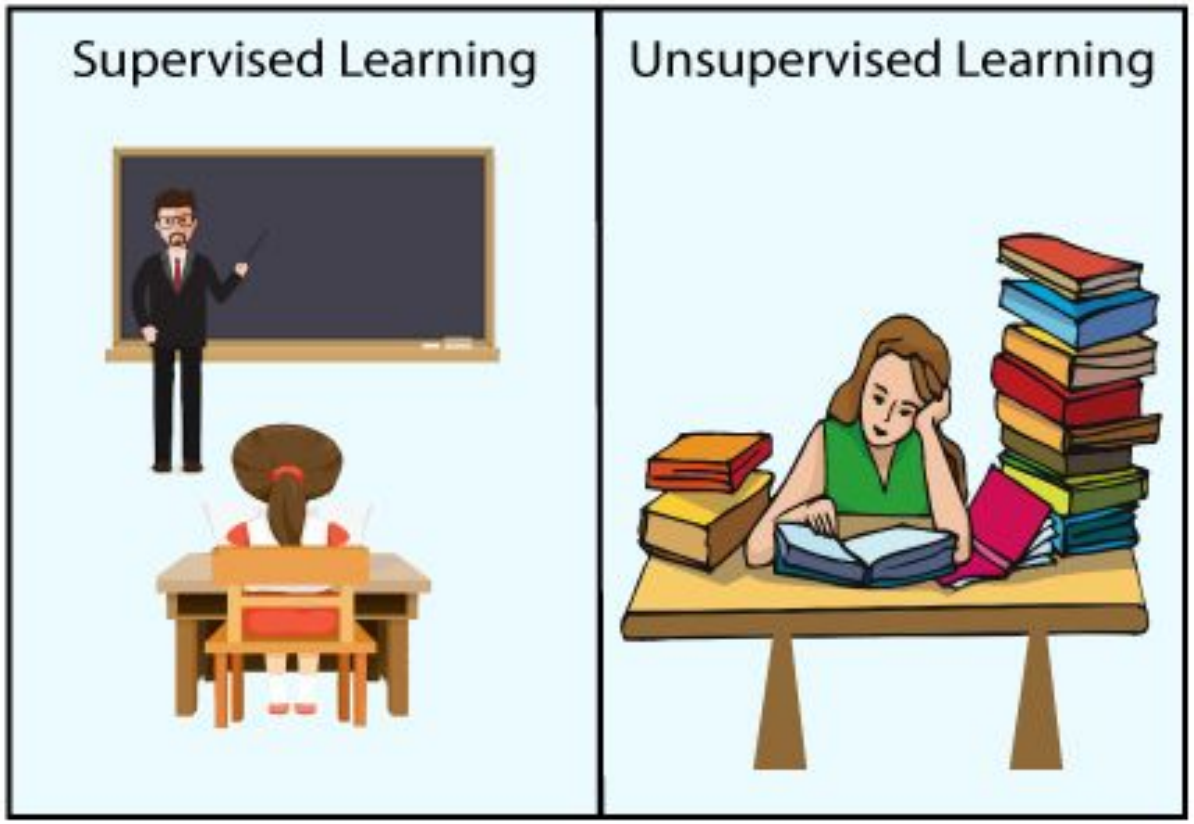
Advantages of Unsupervised Learning

- Unsupervised learning is used for more complex tasks as compared to supervised learning because, in unsupervised learning, we don't have labeled input data.
- Unsupervised learning is preferable as it is easy to get unlabeled data in comparison to labeled data.

Disadvantages of Unsupervised Learning

- Unsupervised learning is intrinsically more difficult than supervised learning as it does not have corresponding output.
- The result of the unsupervised learning algorithm might be less accurate as input data is not labeled, and algorithms do not know the exact output in advance.

Difference between Supervised and Unsupervised Learning



Supervised Learning	Unsupervised Learning
Supervised learning algorithms are trained using labeled data.	Unsupervised learning algorithms are trained using unlabeled data.
Supervised learning model takes direct feedback to check if it is predicting correct output or not.	Unsupervised learning model does not take any feedback.
Supervised learning model predicts the output.	Unsupervised learning model finds the hidden patterns in data.
In supervised learning, input data is provided to the model along with the output.	In unsupervised learning, only input data is provided to the model.
The goal of supervised learning is to train the model so that it can predict the output when it is given new data.	The goal of unsupervised learning is to find the hidden patterns and useful insights from the unknown dataset.
Supervised learning needs supervision to train the model.	Unsupervised learning does not need any supervision to train the model.

Supervised Learning	Unsupervised Learning
Supervised learning can be categorized in **Classification** and **Regression** problems.	Unsupervised Learning can be classified in **Clustering** and **Associations** problems.
Supervised learning can be used for those cases where we know the input as well as corresponding outputs.	Unsupervised learning can be used for those cases where we have only input data and no corresponding output data.
Supervised learning model produces an accurate result.	Unsupervised learning model may give less accurate result as compared to supervised learning.
Supervised learning is not close to true Artificial intelligence as in this, we first train the model for each data, and then only it can predict the correct output.	Unsupervised learning is more close to the true Artificial Intelligence as it learns similarly as a child learns daily routine things by his experiences.
It includes various algorithms such as Linear Regression, Logistic Regression, Support Vector Machine, Multi-class Classification, Decision tree, Bayesian Logic, etc.	It includes various algorithms such as Clustering, KNN, and Apriori algorithm.

Association Rule Learning

- Association rule learning is a type of unsupervised learning technique that checks for the dependency of one data item on another data item and maps accordingly so that it can be more profitable.
- It tries to find some interesting relations or associations among the variables of dataset.
- It is based on different rules to discover the interesting relations between variables in the database.
- The association rule learning is one of the very important concepts of machine learning, and it is employed in **Market Basket analysis, Web usage mining, continuous production, etc.**
- Here market basket analysis is a technique used by the various big retailer to discover the associations between items.

Association rule learning can be divided into three types of algorithms:

1. **Apriori**
2. **Eclat**
3. **F-P Growth Algorithm**

For example, if a customer buys bread, he most likely can also buy butter, eggs, or milk, so these products are stored within a shelf or mostly nearby. Consider the below diagram:

How does Association Rule Learning work?

Association rule learning works on the concept of If and Else Statement, such as if A then B.

Here the If element is called **antecedent**, and then statement is called as **Consequent**. These types of relationships where we can find out some association or relation between two items is known *as single cardinality*. It is all about creating rules, and if the number of items increases, then cardinality also increases accordingly. So, to measure the associations between thousands of data items, there are several metrics. These metrics are given below:

- **Support**
- **Confidence**
- **Lift**

Support

Support is the frequency of A or how frequently an item appears in the dataset. It is defined as the fraction of the transaction T that contains the itemset X. If there are X datasets, then for transactions T, it can be written as:

$$\text{Supp(X)}=\frac{Freq(X)}{T}$$

Confidence

Confidence indicates how often the rule has been found to be true. Or how often the items X and Y occur together in the dataset when the occurrence of X is already given. It is the ratio of the transaction that contains X and Y to the number of records that contain X.

$$\text{Confidence}=\frac{Freq(X,Y)}{Freq(X)}$$

Lift

It is the strength of any rule, which can be defined as below formula:

$$\text{Lift}=\frac{Supp(X,Y)}{Supp(X)\times Supp(Y)}$$

It is the ratio of the observed support measure and expected support if X and Y are independent of each other. It has three possible values:

- If **Lift= 1**: The probability of occurrence of antecedent and consequent is independent of each other.
- **Lift>1**: It determines the degree to which the two itemsets are dependent to each other.
- **Lift<1**: It tells us that one item is a substitute for other items, which means one item has a negative effect on another.

Types of Association Rule Lerning

Association rule learning can be divided into three algorithms:

Apriori Algorithm

This algorithm uses frequent datasets to generate association rules. It is designed to work on the databases that contain transactions. This algorithm uses a breadth-first search and Hash Tree to calculate the itemset efficiently.

It is mainly used for market basket analysis and helps to understand the products that can be bought together. It can also be used in the healthcare field to find drug reactions for patients.

Eclat Algorithm

Eclat algorithm stands for **Equivalence Class Transformation**. This algorithm uses a depth-first search technique to find frequent itemsets in a transaction database. It performs faster execution than Apriori Algorithm.

F-P Growth Algorithm

The F-P growth algorithm stands for **Frequent Pattern**, and it is the improved version of the Apriori Algorithm. It represents the database in the form of a tree structure that is known as a frequent pattern or tree. The purpose of this frequent tree is to extract the most frequent patterns.

Applications of Association Rule Learning

It has various applications in machine learning and data mining. Below are some popular applications of association rule learning:

- **Market Basket Analysis:** It is one of the popular examples and applications of association rule mining. This technique is commonly used by big retailers to determine the association between items.
- **Medical Diagnosis:** With the help of association rules, patients can be cured easily, as it helps in identifying the probability of illness for a particular disease.
- **Protein Sequence:** The association rules help in determining the synthesis of artificial Proteins.
- It is also used for the **Catalog Design** and **Loss-leader Analysis** and many more other applications.

Apriori algorithm – The Theory

- Three significant components comprise the apriori algorithm. They are as follows.
- Support
- Confidence
- Lift
- This example will make things easy to understand.
- As mentioned earlier, you need a big database. Let us suppose you have 2000 customer transactions in a supermarket. You have to find the Support, Confidence, and Lift for two items, say bread and jam. It is because people frequently bundle these two items together.
- Out of the 2000 transactions, 200 contain jam whereas 300 contain bread. These 300 transactions include a 100 that includes bread as well as jam. Using this data, we shall find out the support, confidence, and lift.

Support

- Support is the default popularity of any item. You calculate the Support as a quotient of the division of the number of transactions containing that item by the total number of transactions. Hence, in our example,
- Support (Jam) = (Transactions involving jam) / (Total Transactions)

= 200/2000 = 10%

Confidence

- Confidence is the likelihood that customers bought both bread and jam. Dividing the number of transactions that include both bread and jam by the total number of transactions will give the Confidence figure.
- Confidence = (Transactions involving both bread and jam) / (Total Transactions involving jam)

  = 100/200 = 50%
- It implies that 50% of customers who bought jam bought bread as well.

Lift

- Lift is the increase in the ratio of the sale of bread when you sell jam. The mathematical formula of Lift is as follows.
- Lift = (Confidence (Jam $\rightarrow$ Bread)) / (Support (Jam))

  = 50 / 10 = 5
- It says that the likelihood of a customer buying both jam and bread together is 5 times more than the chance of purchasing jam alone. If the Lift value is less than 1, it entails that the customers are unlikely to buy both the items together. *Greater the value, the better is the combination.*

How does the Apriori Algorithm in Data Mining work?

- We shall explain this algorithm with a simple example.
- Consider a supermarket scenario where the itemset is I = {Onion, Burger, Potato, Milk, Beer}. The database consists of six transactions where 1 represents the presence of the item and 0 the absence.

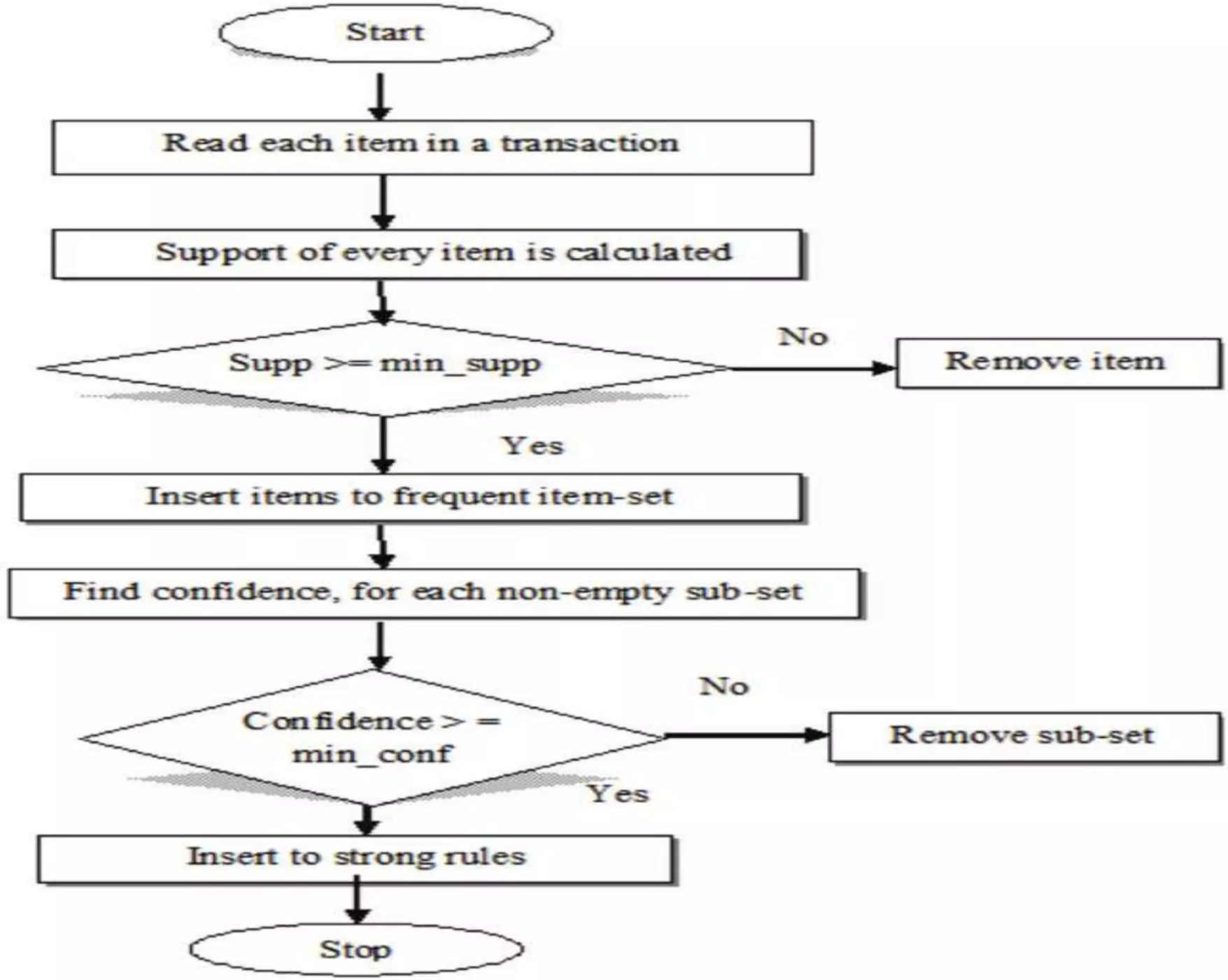
Contd...

Transaction ID	Onion	Potato	Burger	Milk	Beer
t_1	1	1	1	0	0
t_2	0	1	1	1	0
t_3	0	0	0	1	1
t_4	1	1	0	1	0
t_5	1	1	1	0	1
t_6	1	1	1	1	0




The Apriori Algorithm makes the following assumptions

- All subsets of a frequent itemset should be frequent.
- In the same way, the subsets of an infrequent itemset should be infrequent.
- Set a threshold support level. In our case, we shall fix it at 50%


Start
Read each item in a transaction
Support of every item is calculated
Supp >= min_supp
No
Remove item
Yes
Insert items to frequent item-set
Find confidence, for each non-empty sub-set
Confidence > = min_conf
No
Remove sub-set
Yes
Insert to strong rules
Stop

Steps

- Create a frequency table of all the items that occur in all the transactions. Now, prune the frequency table to include only those items having a threshold support level over 50%.
- We arrive at this frequency table.

Threshold Support >= 50%

Step 1 : One Itemset
C1

Itemset	Frequency
Onion	4
Potato	5
Burger	4
Milk	4
Beer	2

Pruning

Step 2
F1

Itemset	Frequency
Onion	4
Potato	5
Burger	4
Milk	4

Make pairs of items such as OP, OB, OM, PB, PM, BM. This frequency table is what you arrive at.

Step 3 : Two Itemset C2

Itemset	Frequency
Onion Potato	4
Onion Burger	3
Onion Milk	2
Potato Burger	4
Potato Milk	3
Burger Milk	2

Pruning

Apply the same threshold support of 50% and consider the items that exceed 50% (in this case 3 and above).
Thus, you are left with OP, OB, PB, and PM

Step 4 F2

Itemset	Frequency
Onion Potato	4
Onion Burger	3
Potato Burger	4
Potato Milk	3

Note: Consider items from F1 only for making C2

Look for a set of three items that the customers buy together. Thus we get this combination.
OP and OB gives OPB
PB and PM gives PBM

Step 6 : Three Itemset C3

Itemset	Frequency
Onion Potato Burger	3
Potato Burger Milk	2

Pruning

Determine the frequency of these two itemsets. You get this frequency table.
If you apply the threshold assumption, you can deduce that the set of three items frequently purchased by the customers is OPB.
We have taken a simple example to explain the apriori algorithm in data mining. In reality, you have hundreds and thousands of such combinations.

Step 7 F3

Itemset	Frequency
Onion Potato Burger	3

Note: Consider items from F2 only for making C3

Apriori Algorithm – Pros

- Easy to understand and implement
- Can use on large itemsets

Apriori Algorithm – Cons

- At times, you need a large number of candidate rules. It can become computationally expensive.
- It is also an expensive method to calculate support because the calculation has to go through the entire database.

Apriori Algorithm – Limitations

- The process can sometimes be very tedious.

Example:

TID	List of item_IDs
T100	I1, I2, I5
T200	I2, I4
T300	I2, I3
T400	I1, I2, I4
T500	I1, I3
T600	I2, I3
T700	I1, I3
T800	I1, I2, I3, I5
T900	I1, I2, I3

- Scan D for count of each candidate
  - C1: I1 – 6, I2 – 7, I3 -6, I4 – 2, I5 - 2
- Compare candidate support count with minimum support count (min_sup=2)
  - L1: I1 – 6, I2 – 7, I3 -6, I4 – 2, I5 - 2
- Generate C2 candidates from L1 and scan D for count of each candidate
  - C2: {I1,I2} – 4, {I1, I3} – 4, {I1, I4} – 1, …
- Compare candidate support count with minimum support count
  - L2: {I1,I2} – 4, {I1, I3} – 4, {I1, I5} – 2, {I2, I3} – 4, {I2, I4} - 2, {I2, I5} – 2
- Generate C3 candidates from L2 using the join and prune steps:
  - Join: C3=L2xL2={{I1, I2, I3}, {I1, I2, I5}, {I1, I3, I5}, {I2, I3, I4}, {I2, I3, I5}, {I2, I4, I5}}
  - Prune: C3: {I1, I2, I3}, {I1, I2, I5}
- Scan D for count of each candidate
  - C3: {I1, I2, I3} - 2, {I1, I2, I5} – 2
- Compare candidate support count with minimum support count
  - L3: {I1, I2, I3} – 2, {I1, I2, I5} – 2
- Generate C4 candidates from L3
  - C4=L3xL3={I1, I2, I3, I5}
  - This itemset is pruned, because its subset {{I2, I3, I5}} is not frequent => C4=null